

PRAKTIKUM SISTEM OPERASI

MODUL 11

Memori Xinu



Oleh:

Dimastian Aji Wibowo

2311104058

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

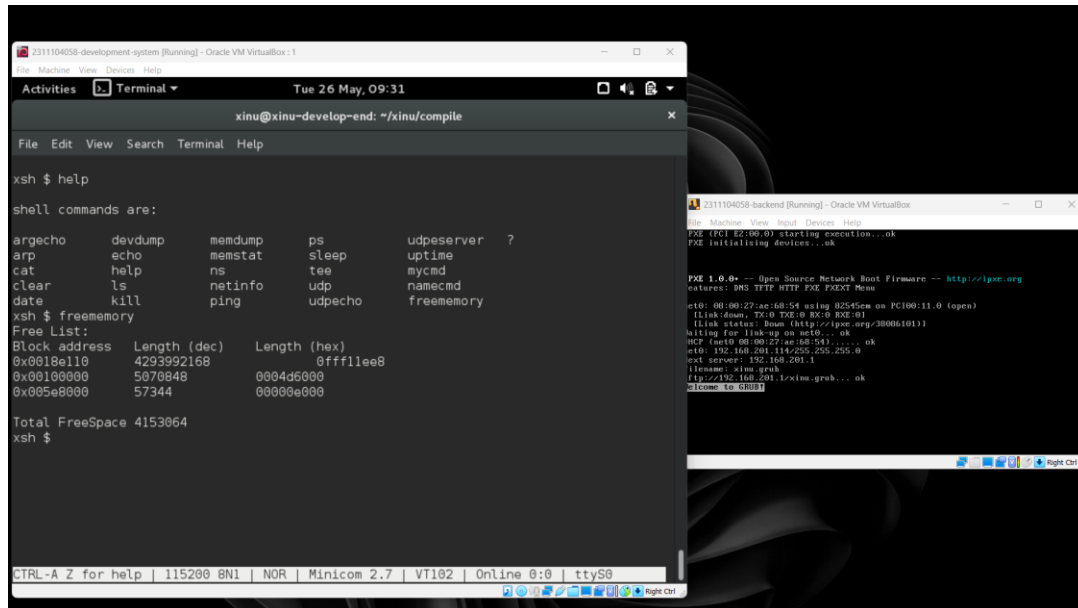
DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

JURNAL

1. Perintah freememory



```
xsh $ help
shell commands are:
argecho    devdump    memdump    ps          udpserver  ?
arp        echo       memstat    sleep       uptime
cat        help      ns         tee         mycmd
clear      ls         netinfo    udp         namecmd
date       kill      ping       udpecho    freememory
xsh $ freememory
Free List:
Block address  Length (dec)  Length (hex)
0x0010e110    4293992168   0ffff1ee8
0x00108000    5070848     0004d6000
0x005e8000    57344       00000e000
Total FreeSpace 4153064
xsh $
```

2. Pertanyaan

a. Mengapa Xinu memisahkan data segment dan BSS segment?

Xinu memisahkan data segment dan BSS segment berdasarkan status inisialisasi nilainya untuk mencapai efisiensi dalam manajemen memori image. Secara spesifik, data segment bertugas menyimpan semua variabel global yang nilainya telah diinisiasi secara eksplisit sejak awal program berjalan. Sebaliknya, BSS segment dikhususkan untuk menampung variabel global yang belum atau tidak diinisiasi nilainya. Pemisahan fungsi ini sangat penting karena sistem operasi menjadi tahu persis bagian memori mana yang benar-benar membutuhkan alokasi memori sekaligus penyimpanan nilai bawaan, dan bagian mana yang sekadar membutuhkan penyediaan ruang kosong saja, sehingga memori terkelola secara optimal.

b. Bagaimana alokasi dan dealokasi memori selama eksekusi memengaruhi ukuran free space?

Selama proses eksekusi berlangsung, aktivitas alokasi dan dealokasi memori sangat memengaruhi sisa kapasitas free space. Pada saat sistem melakukan booting atau saat sebuah proses baru diciptakan, proses tersebut akan

dialokasikan ke dalam free space yang tersedia, sehingga secara otomatis menempati dan mengurangi ukuran ruang kosong. Namun, ketika proses tersebut telah selesai dieksekusi atau mengalami terminasi, alokasi memori yang sebelumnya dipinjam akan dihapus (didealokasi) dan dikembalikan ke sistem, sehingga ukuran free space bertambah kembali. Siklus masuk dan keluarnya proses ini terus terjadi, membuat ukuran free space menjadi sangat dinamis bergantung pada tingkat kerumitan program yang sedang berjalan.

- c. Mengapa penggunaan heap lebih berpotensi menimbulkan masalah dibandingkan stack?

Penggunaan memori heap jauh lebih berpotensi menimbulkan masalah dibandingkan memori stack karena adanya perbedaan mendasar dalam sistem dealokasinya. Pada struktur stack, alokasi dan dealokasi berjalan sangat terstruktur menggunakan prinsip Last-In-First-Out (LIFO), di mana memori dijamin pasti akan dihapus secara otomatis saat sebuah proses telah selesai. Sebaliknya, manajemen memori pada heap diserahkan sepenuhnya kepada pengguna. Ketika proses selesai, memori heap tidak terhapus dengan sendirinya, sehingga jika pengguna lupa menghapusnya secara manual dan eksplisit, sistem akan mengalami penumpukan sampah memori yang berujung pada kebocoran memori (memory leak).

- d. Mengapa Xinu menggunakan struktur linked list untuk menyimpan block?

Xinu memilih menggunakan struktur linked list untuk mengelola memori karena sistem dituntut untuk bisa menyimpan sekaligus melacak informasi detail mengenai posisi semua free block (memori kosong) yang tersebar. Melalui sebuah variabel global bernama memlist, sistem memiliki pointer (penunjuk) ke blok memori kosong yang pertama. Blok pertama ini kemudian saling terhubung dan menunjuk ke blok kosong berikutnya secara berantai, lengkap dengan informasi letak alamat serta ukuran setiap blok. Mekanisme pencatatan berantai ini sangat efisien dan memudahkan Xinu saat harus mencari ruang kosong, mengalokasikan memori untuk proses baru, sekaligus memperbarui daftar sisa memori seketika.

- e. Apa tantangan utama dalam penggunaan heap di Xinu?

Tantangan paling utama dalam penggunaan heap di ekosistem Xinu adalah besarnya beban tanggung jawab manajemen yang dilimpahkan langsung kepada pengguna. Penghapusan sisa memori heap wajib dilakukan secara manual dan eksplisit oleh proses yang memanggilnya, sehingga sangat rentan mengalami memory leak akibat kelalaian manusia. Selain itu, sistem juga dihadapkan pada risiko kelebihan batas, di mana besaran permintaan alokasi memori heap bisa melampaui sisa free space yang sesungguhnya tersedia. Tantangan besar lainnya adalah heap rentan memicu terjadinya fragmentasi, yaitu kondisi memori yang terpecah-pecah menjadi blok-blok berukuran kecil seiring berjalannya waktu alokasi dan dealokasi, sehingga menyulitkan sistem mencari ruang kosong berukuran besar.